

ft_acki

Veni, vidi, converti

Summary:
In this exercise, you will convert a decimal number into Roman numerals, without the help of any conversion table.

Version: 1.0

Contents

I Introduction **2**

II Common Instructions **3**

III AI Instructions **4**

IV Mandatory part **5**

IV.1 Technical considerations 5

IV.2 Roman numerals 101. 6

IV.3 Expected output. 6

IV.4 Examples. 7

V Submission and peer-evaluation **8**

Chapter I

Introduction

Rome, 44 BC. A strange traveller, wearing a hoodie with a big "42" on it, has just left an even stranger machine on the steps of the Senate. It glows, it hums, and it only speaks in Arabic numerals, a notation that will not reach Europe for another twelve centuries.

Julius Caesar does not like things he cannot read. He orders his scribes to translate every number displayed by the machine into proper Roman numerals. Unfortunately, the scribes' precious conversion scrolls have just been stolen. Rumour has it that Brutus did it.

No scroll, no table, no dictionary: the scribes will have to understand how Roman numerals are actually built. And so will you.

Hurry up: the Ides of March are coming.



Fun fact: Roman numerals are still everywhere today: on clock faces, in the names of kings and popes, in chapter numbers... and in the chapters of this very subject.

Chapter II

Common Instructions

- This exercise must be written in C.
- Your code must be written in accordance with the Norm. It will be checked with `norminette`, and you will receive a 0 if there is a Norm error.
- Your program must compile with `cc` and the flags `-Wall`, `-Wextra` and `-Werror`. If it does not compile, you will get 0.
- Your program must not quit unexpectedly (segmentation fault, bus error, double free, etc.). If this happens, your exercise will be considered non-functional and will receive a 0.
- Declaring global variables is strictly forbidden.
- You may only use the external functions listed in this subject. Using a forbidden function is considered cheating. Cheaters get -42, and this grade is non-negotiable.
- All your output must be done with `write`: `printf`, `fprintf`, `putchar` and the like are forbidden in every exercise.
- No Makefile is required: your program will be compiled with the command given in Chapter IV.
- We encourage you to test your program with many different inputs, including edge cases (0, 3999, 4000, huge numbers, letters, signs...). Your evaluators will do it, for sure.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded.

Chapter III

AI Instructions

Context

This exercise is designed to help you consolidate the fundamental building blocks of your 42 training. To properly anchor these skills, you need to struggle with them yourself: true learning requires genuine intellectual effort, through challenge, repetition and peer-learning exchanges.

Main message

- ☛ Build strong foundations without shortcuts.
- ☛ The learning journey is more important than the result.
- ☛ An exercise solved by an AI is an exercise you did not solve.

Learner rules

- You should apply reasoning to your exercise, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You must be able to explain every line you submit.

✓ Good practice:

> I write the numbers from 1 to 9 in Roman numerals on a sheet of paper, then 10 to 90, then 100 to 900. I put the three lists side by side and notice something. I get it.

✗ Bad practice:

> I ask an AI to convert numbers into Roman numerals. It gives me a beautiful conversion table, which is forbidden. I fail. During the exam (no AI) I'm stuck again. I fail.

Chapter IV

Mandatory part

Turn-in directory	`ex02/`
Files to Submit	`ft_acki.c`
Compilation	`cc -Wall -Wextra -Werror ft_acki.c`
Arguments	`A number between 1 and 3999`
External Function	`write`
Description	`Converts a decimal number into Roman numerals.`

IV.1 Technical considerations

✓ **Allowed:**

- The `write` function.
- Arithmetic operators (`/` and `%` are your best friends), conditions, loops, and helper functions within the limits of the Norm.
- Roman letters written directly in your code, as character literals: `'I'`, `'V'`, `'X'`...

✗ **Forbidden:**

- Any kind of conversion table or dictionary: no array, no string and no structure storing Roman symbols or their values, such as `int values[] = {1000, 900, 500, ...}` or `"IVXLCDM"[i]`.
- Any other external function (`printf`, `atoi`...).
- Global variables.



`Your program must compute the Roman numeral, not look it up. If your evaluator finds a conversion table in your code, whatever its form, the exercise is failed.`

IV.2 Roman numerals 101

Roman numerals use seven symbols:

Symbol	I	V	X	L	C	D	M
Value	1	5	10	50	100	500	1000

- Symbols are written from the largest to the smallest, and their values are added: `VIII` = 5 + 1 + 1 + 1 = 8, `MDCLXVI` = 1666.
- The same symbol is never repeated more than three times in a row: 4 is not written `IIII`.
- Instead, a smaller symbol placed just before a larger one is subtracted. Only six such pairs exist: `IV` = 4, `IX` = 9, `XL` = 40, `XC` = 90, `CD` = 400 and `CM` = 900.
- There is no zero, and the largest number that can be written with these rules is 3999: `MMMCMXCIX`.



`1987 = 1000 + 900 + 80 + 7, so 1987 is written M + CM + LXXX + VII, that is MCMLXXXVII.`

`Now write 7, 70 and 700 in Roman numerals. Then 4, 40 and 400. Do you see a pattern?`

IV.3 Expected output

- Your program takes exactly one argument: a number made only of digits (no sign, no space), between 1 and 3999. Leading zeros are allowed: `0042` is 42.
- It displays the Roman numeral of this number in uppercase, followed by a newline, on the standard output, and returns 0.

Otherwise, it displays one of the following messages on the **error output**, followed by a newline, and returns 1. The checks are made in this order:

Situation	**Message**
wrong number of arguments	`Error: usage ./a.out <number>.`
empty, or not only digits	`Error: arg must contain only digits.`
0, or greater than 3999	`Error: arg must be between 1 and 3999.`



`Be careful with very long numbers, such as 99999999999999999999: your program must display the range error, not crash or display something random.`

IV.4 Examples

```
$> ./a.out 42 | cat -e
XLII$
$> ./a.out 4 | cat -e
IV$
$> ./a.out 1987 | cat -e
MCMLXXXVII$
$> ./a.out 2026 | cat -e
MMXXVI$
$> ./a.out 3999 | cat -e
MMMCMXCIX$
$> ./a.out 0042 | cat -e
XLII$
$>
```

Errors are displayed on the error output, so they are not affected by `| cat -e`:

```
$> ./a.out 0
Error: arg must be between 1 and 3999.
$> ./a.out 4000
Error: arg must be between 1 and 3999.
$> ./a.out 99999999999999999999
Error: arg must be between 1 and 3999.
$> ./a.out -12
Error: arg must contain only digits.
$> ./a.out 12abc
Error: arg must contain only digits.
$> ./a.out
Error: usage ./a.out <number>.
$> echo $?
1
$>
```

Chapter V

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer evaluation. Make sure to double-check the names of your files and folders to ensure they are correct.



`Your file must be placed in a folder named ex02 at the root of your repository: ex02/ft_acki.c`

During the evaluation, a brief **modification of the exercise** may be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

This step is meant to verify your actual understanding of the exercise. It should be feasible within a few minutes, in any development environment you choose.

For this exercise, your evaluator can for example ask you to:

- display the result in lowercase: `xlii`;
- display the details of the conversion: `1987 = M + CM + LXXX + VII`.